Ctrl-F: Force as an Interface for Humanoid Loco-Manipulation

Anonymous Author(s)

***Abstract*— Humanoid motion tracking has scaled to general policies that reproduce diverse human motion, but the resulting behavior is in free-space: the tracker knows where the body should be, not how it should interact with the environment. Other work makes the object part of the objective, yet abstracts that interaction to contact or geometry, which leaves the demanding cases under-explored. We argue that force is the natural command channel missing between motion imitation and object interaction, the signal that says not just where to make contact, but how hard. We present Ctrl-F, a hierarchical motion–force framework that augments motion tracking with an explicit force command, a 3-D force at each designated contact body such as a hand or a foot. The hierarchical framework comprises: (1) A unified motion–force controller tracks the reference motion while applying the commanded force to the world. (2) A high-level force planner that issues a force command online based on object state and proprioception observations. In simulation and on hardware, Ctrl-F completes loco-manipulation that demands large, sustained interaction force and adjusts its command online as the load changes, both out of reach of a force-blind tracker. On hardware it drags, pushes and raises with the hands and kicks with a foot, reaching $20\,$kg on the pushed and the kicked box where the baseline moves nothing loaded, and it keeps dragging when weight is added mid-task. Code will be released upon acceptance.**

I. INTRODUCTION

Many everyday whole-body tasks involve sustained contact with the environment: we carry a heavy box across a room, drag a crate, push a door that pushes back. These actions involve rich whole-body motion together with continuous physical interaction, and the interaction changes as the object responds. A heavier box must be gripped harder; a crate that gets stuck must be pulled harder.

Humanoid robots have learned to reproduce human motion. Policies have evolved from single clips [1] to trackers that cover hours of motion capture with one model [3], [4], [5], [6], inheriting the scalability of motion data itself. But a reference motion prescribes only where the body should be, and the behavior it produces is in free-space. When interacting with an object, a strong tracker can fail silently: in our experiments, asked to raise a box drawn between 1.5 and $6\,$kg with no force channel, it leaves the box just staying there sliding down from the palm and completes none of its episodes. The motion is right, but the interaction is missing.

Recent work has since brought the object into the objective, using representations such as direct object motion, contact label, and distance fields [8], [9], [10]. These abstractions are exactly what lets them scale. They specify, however, whether and where contact happens, not how hard. The policy cannot be told to press harder, and cannot adapt when the same motion must move a much heavier load. Force is the natural signal for these cases. It is what the object actually experiences, and is a unified representation to describe such interaction. Also, unlike contact label, force have magnitude and direction, making it suitable to serve as a continuous command rather than merely a description.



Fig. 1. Ctrl-F on hardware, running the raise-walk-kick sequence with a $1\,$kg box in the hands and an $8\,$kg cart to kick. Raising the box is already a force task: the hands slide off a box they do not press into, and a tracker without a force command does not lift it at all.

Force has been brought into humanoid control before, though not yet in that role: as an external disturbance the tracker learns to resist under a force curriculum [13], as end-effector stiffness modulated through hindsight goal perturbation [14], and as a commandable position–force target at a single interaction point [15]. Each provides part of the required capability. What remains missing is a controller that preserves the full motion-imitation representation, accepts explicit force commands, *and* adaptively selects those forces based on the object's response.

To close that gap, we present **Ctrl-F**, a hierarchical humanoid loco-manipulation framework that explicitly commands the forces exerted by the robot on the environment. The low level is a general motion–force controller that consumes a reference motion *and* a 3-D force command per designated contact body, trained object-free by *inverse-force training*. The tracker keeps the scalability of a general motion tracker, needing no object or paired interaction data, yet exposes force as an independently commandable input. On top of the frozen tracker, a high-level force planner chooses force command from object state and proprioception. It learns on-policy from an analytical force model and improves through reinforcement learning, so the commands adapt to contact dynamics the analytical model does not capture.

Anonymized for double-blind review.

Our contributions are:

1) We define a general force interface for humanoid interaction and train it without objects. The robot's interaction with the environment is specified as a 3-D force at each designated contact body. The motion–force tracker is trained without the need for paired object assets or interaction data. It serves different objects and interaction patterns without task-specific representations.
2) We train a high-level force planner that can predict the desired force command based on the object and robot state feedback. The resulting policy requires neither measured force nor object mass at deployment, yet its force commands still adapt online to object state and load.
3) We demonstrate sim-to-real deployment of our framework on a Unitree G1 humanoid robot across multiple physical interaction tasks and loading scenarios, including online adaptation to load changes during execution.

II. RELATED WORK

Humanoid motion tracking. DeepMimic established pose- and velocity-imitation rewards with early termination and reference-state initialization [1]. AMASS-scale data [2] enabled general trackers, including BeyondMimic [3], SONIC [4], GMT [5] and Humanoid-GPT [6], and H2O applies a tracker as a real-time teleoperation backbone [7]. These policies reproduce a kinematic trajectory and are scored on their fidelity to it. Because the scene enters neither the objective nor the metric, the behavior they learn is free-space: it is defined without reference to anything the body might touch, and a tracker can reproduce a motion perfectly while the world it was meant to act on goes untouched. We build directly on this line, adding the missing interaction capability rather than replacing it.

Humanoid object interaction. A second body of work takes the object into account. HDMI co-tracks humanoid and object trajectories extracted from video [8]; SceneBot conditions a single general tracking policy on per-link contact labels, with scene assets reconstructed from human motion [9]; LessMimic uses unified distance-field representations for long-horizon, reference-free interaction [10]. Each controls interaction through a quantity that is geometric: where the object should be, which links should touch it, how near the surfaces are. These are exactly the abstractions that let the methods scale to many scenes, and they are silent on magnitude. A contact label specifies only whether contact occurs, not its magnitude. The policy cannot be asked to press harder, nor to adapt when the same motion must move a heavier load.

Force control, classical and learned. Impedance control [11] and hybrid position/force control [12] supply the mechanical vocabulary for regulating contact, but not a recipe for preserving a high-dimensional motion capture reference on a floating-base humanoid whose contacts change during the motion. Among learned approaches, FALCON trains dual lower/upper-body agents under a torque-limit-aware external-force curriculum, making end-effector tracking robust to

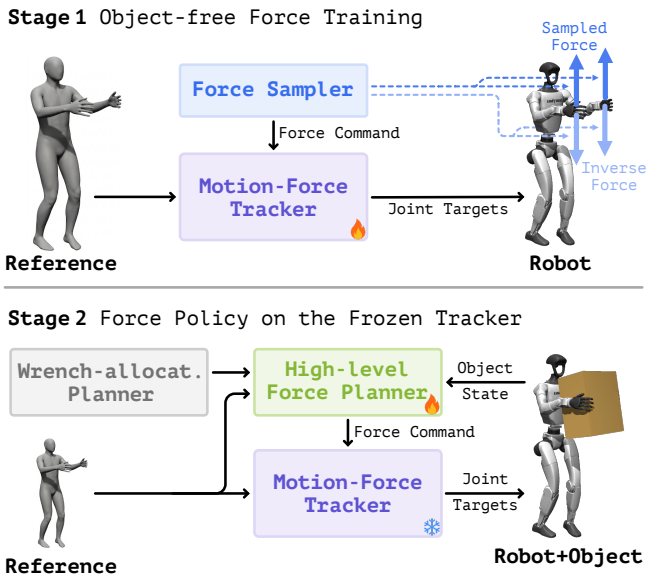


Fig. 2. Ctrl-F. *Stage 1* trains the motion–force tracker without objects: a force command is randomly sampled and its corresponding inverse force is applied to the robot, teaching the tracker to follow its reference while exerting the commanded load. *Stage 2* freezes that tracker and learns the force command itself from object and robot state, supervised by the analytical force model and refined by reinforcement learning.

applied wrenches [13]. FALCON treats force as a disturbance the policy is hardened against, rather than a quantity specified by the task. CHIP modulates end-effector stiffness on a generalist tracker through hindsight goal perturbation [14], the resulting force is realized indirectly, as stiffness times a reference offset, and no measurement of the object closes the loop around it. UniFP unifies position and force control at a single interaction point without force sensors, under a diffusion imitation high level trained from teleoperated, force-annotated data [15]; force is commandable there, but the high level reproduces demonstrated force patterns rather than selecting them from the object's response, and its interface is an end-effector target rather than a whole-body motion reference. These approaches leave two capabilities unaddressed. Force control is either not integrated with a general motion-tracking representation and thus does not extend to the breadth of motion covered by general trackers, or the applied force is determined in advance rather than adapted online to the object's response. Ctrl-F integrates general motion tracking with adaptive force selection.

III. METHOD

We propose a hierarchical framework (Fig. 2) comprising a low-level motion-force hybrid tracker and a high-level force planner. The low-level tracker takes a robot reference motion, a 3-D force command per designated contact body, and proprioception as input, and outputs full-body joint-position targets. It has no notion of an object at all: the force command is simply another input to track, allowing it to be trained without an object. The high-level force planner takes as input the reference motions of both the robot and the object, the desired contact point on the object, the object's

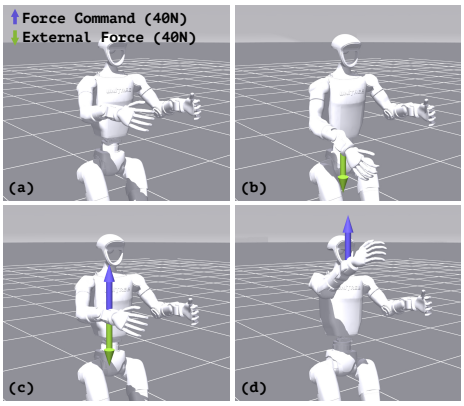


Fig. 3. Effect of force commands during a static hold with a fixed reference posture.

measured state, and proprioception, and outputs the desired force command for the low-level tracker. Proprioception and object state close the loop: by watching how the object and robot body actually move against where the reference says it should be, the high-level force planner decides how hard to ask the low-level tracker to push, grip, or pull.

The force command is identical across tasks: one 3-D force vector per designated contact body, the hands and a foot in our tasks, and, in principle any body. The policy's action space remains fixed across tasks, allowing a single high-level force planner to span different mechanics. The two levels are trained sequentially: the low-level tracker is trained without an object and then frozen, so that learning object interaction never requires relearning whole-body balance.

A. Low-level motion–force tracker

The low-level policy is a motion tracker with one addition: a contact-force command input. Its objective is to maintain whole-body tracking while executing the force command.

Inverse-force training. The difficulty is that no object is present during low-level training: a commanded force has nothing to act on. *Inverse-force training* resolves this by turning force execution into a tracking problem. Each reference clip carries a per-body load mask marking when each contact body is free and can exert force. Whenever that mask marks a body as free, we sample a force command and apply the *opposite* wrench to that body while retaining the standard motion-tracking objective. To track the reference motion under this external wrench, the policy must produce an equal and opposite force, precisely the commanded force. In this way, the policy learns to map a force command to the corresponding force-producing behavior without requiring an object during training. Because the learned response depends only on proprioception rather than object-specific observations or contact modeling, it transfers naturally from compensating for the injected wrench to exerting force through physical object contact.

Fig. 3 illustrates the learned force response on a static reference posture. Every panel in the figure is the trained motion-force tracking policy rollout in close loop, showing exactly the controller's behavior after training. Without an external wrench or force command, the policy holds the reference posture (a). An external wrench alone drives the arm away from the reference (b), while adding the matched force command causes the policy to counteract the wrench and recover the reference posture (c). With the command alone and no opposing wrench as in (d), the policy responds to the force command by producing that force against the world, and with nothing external to resist it, such action drives the robot body away from the reference motion and towards the force command direction, by approximately the same amount as in the opposite direction under the wrench alone. Together, these cases show that the policy has learned the force response induced by inverse-force training.

Load mask. As mentioned previously, we use a per-body load mask to determine where and when force commands may be sampled. Importantly, the mask encodes physical feasibility rather than task semantics: it does not identify when the reference motion is interacting with an object, but only whether a body is in a configuration where it can meaningfully exert an external force. For example, a supporting foot is masked out because it cannot independently exert the commanded interaction force, whereas the hands can remain available for force commands throughout most reference motions. This label can be obtained by simple motion-processing, therefore we can use the same corpus economics as a general motion tracker.

Curriculum. We design curriculum for learning motion-force tracking during the low-level training. Each contact body has a fixed range its force command is drawn from. During training, we first fix the force command to zero, hence no force is asked of the policy and it learns the basic motion-tracking ability. Starting from a given training step, we gradually ramps the force command sampling scale to one, making the policy meets small commanded forces first and the full per-body range last.

B. High-level force planner

In the second stage, the real object is put into the scene, and the high-level force planner is trained to give force command to the frozen low-level motion–force controller. That force is not observable from the reference motion: the same clip carried out on a light box and a heavy one needs different forces. However, learning it from reward alone is slow and unreliable, because a force command is only scored through its effect on the object, and most commands break contact before they produce a stable one. We therefore use an analytical force model as a teacher, and train the high-level force planner with an imitation loss on top of PPO.

Analytical force model supplies the force targets. The high-level force planner needs a source of physically sensible force targets. We derive it from the object alone. We therefore use an analytical force model to give the coarse force targets. The analytical force model first computes the spatial wrench $W^\star$ the object needs by computed-wrench control on the object: proportional–derivative feedback on object pose and velocity error, mapped through the object's Newton–Euler

dynamics, plus feedforward gravity compensation for airborne objects and Coulomb-friction compensation for objects on the ground. It then allocates that wrench to the available contact-force channels:

$$\text{find } f \quad \text{such that} \quad Gf \approx W^{\star}, \quad f \in \mathcal{F}, \tag{1}$$

Here $W^{\star} \in \mathbb{R}^6$ is the desired object wrench just computed, f stacks the contact forces the robot may apply at its designated contact bodies, G maps those forces to the wrench they produce on the object, and $\mathcal{F}$ is the set of forces that channel can physically carry. A task enters this problem only as a triple $(G, \mathcal{F}, \text{load})$ and differs in nothing else. Every allocation is solved in closed form and batched across environments, allowing the analytical force model to be queried at every step of every environment during training.

The analytical force model reads privileged object state and true mass, hence it is not a controller we could deploy. Also, it does not need to be accurate. It is coarse guidance and not a target: it only has to put the learner in the right region of the command space, reinforcement learning is free to leave it wherever the model is wrong, and that is why a small set of interaction patterns is enough.

Annealed imitation on top of PPO. The high-level force planner is trained against two signals at once. The analytical force model is queried at the states the current policy visits, in the style of DAgger [17], keeping supervision on-policy. Its weight in the loss decays over training, following DAPG [16], allowing the policy to depart from the model's command wherever departure pays. The reinforcement objective is scored on the object rather than on the robot: reward follows the object's reference and success is defined on the object, not on how closely the robot matches the clip; the frozen tracker already handles that. During training, the object mass are sampled within a range at the beginning of the episodes, and will also change inside the episodes, making the high-level force planner learn the adaptation.

IV. EXPERIMENTS

Our experiments answer four questions in simulation and one on hardware. **Q1:** does the low-level motion–force controller realize an explicit force interface, and at what cost to tracking? **Q2:** does the hierarchy complete forceful tasks, and how does it scale with object mass relative to force-blind controllers? **Q3:** does the commanded force adapt online when the load changes? **Q4:** is model-supervised BC+RL the right training recipe? **Q5:** does the full hierarchy transfer to hardware?

A. Experimental setup

Task families. Our tasks fall into three families by which bodies carry the load (Fig. 4). *Hand tasks* raise, push, drag, lift and hold a box with the hands. *Foot tasks* kick and sweep with one foot planted. *Composite tasks* carry a box in the hands while kicking or sweeping with a foot, loading wrists and an ankle at once. Each family has its own motion corpus and its own controller pair, one low-level and one high-level shared by every task in the family; a result within a family is therefore one policy across several tasks rather than one policy per task.

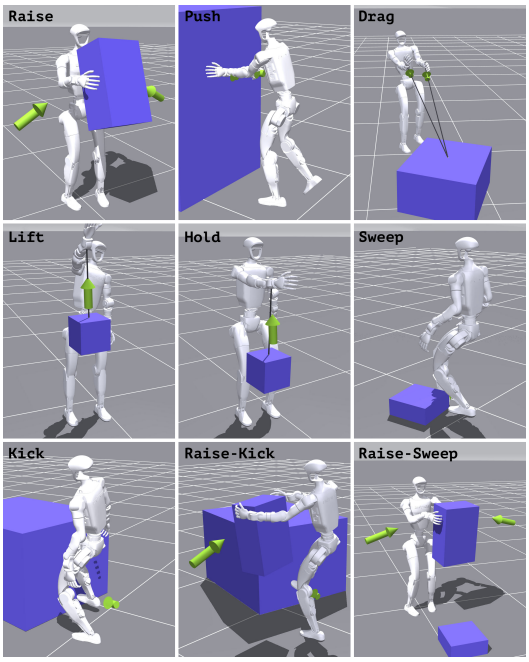


Fig. 4. The nine simulated tasks, in the reading order of the three families: hand tasks raise, push, drag, lift and hold; foot tasks sweep and kick; composite tasks carry a box while kicking or sweeping. Green arrows indicate the commanded force at each designated contact body, as specified by the high-level force planner and realized by the low-level controller. A shared action space spans all nine tasks, enabling a single frozen tracker to generalize across the entire task family.

TABLE I. denotes the range sampled during evaluation; friction denotes the coefficient at the task-relevant contact surface: the palms for the raised box and the floor for all other tasks.

Object	Mass (kg)	Friction
Raised box	1.8–6.0	0.60
Tethered box	3.6–7.0	—
Kicked box	3.0–6.0	0.25
Pushed box	4.6–13.0	0.38
Dragged crate	7.0–8.8	0.57

Objects. Table I gives the objects the tasks to act on. Mass is randomized per episode within a band, and the sweep of Sec. IV-C reports it as a multiple of each band's midpoint, so $1.0\times$ means the same relative position in every band even though the absolute weights differ by a factor of two across objects.

Baseline. We compare against SONIC [4], a general humanoid motion tracker released with public G1 weights, run inside our simulator, mjlab [18], on the same clips and scored by the same metrics. It is the right comparison for this paper because it embodies the free-space position: a strong tracker with no force channel at all. Where a released checkpoint cannot be run, as on the interaction tasks that need a force command, the comparison is instead against our own policy with that command removed.

Metrics. We report motion tracking in two frames throughout. *Global* error compares body positions to the reference in the world frame, and therefore accumulates drift;

TABLE II. Q1(a): Tracking with the force command zeroed and no external load, averaged across all twelve clips of the three task classes. Errors are in cm; "forced" denotes the bodies specified by each clip's load mask. Local errors are root-anchored and global errors are measured in the world frame. Each class uses its own tracker; results are therefore averaged across clips, with the arms paired within each clip.

	Ours	SONIC
Forced-body error, local	**2.96**	5.89
Forced-body error, global	**5.39**	9.40
Whole-body error, local	**3.10**	3.75
Whole-body error, global	**4.98**	9.00
Completion (%)	**100.0**	80.2

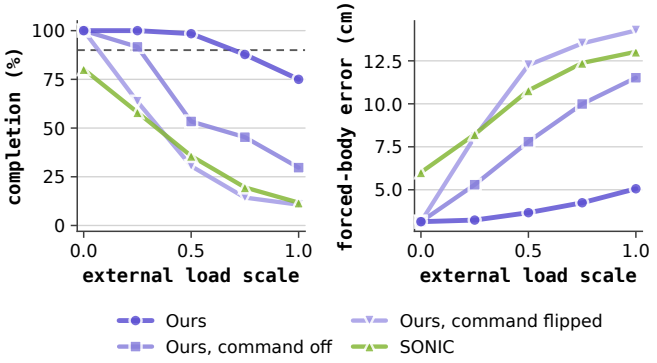


Fig. 5. Q1(b): completion and forced-body error as the replayed wrench is scaled, averaged over all twelve clips. One trace per clip is replayed identically to the four arms and scaled together with the command each is given, leaving the arms to differ only in what the policy is told; the three command variants are one policy and coincide at zero load. The scale is the fraction of the per-body command range the wrench uses, so 1.0 means the same thing on a wrist (60 N) and an ankle (30 N) and nothing plotted is out of distribution. Dashed line is the 90% completion level. Errors are over all active steps, a convention that flatters an arm terminating early.

local error re-anchors the reference to the robot's own root before comparing, and therefore measures posture fidelity independently of where the robot ended up. We report each for the whole body and, separately, for the contact bodies that carry force: the bodies the interface acts on are exactly where a force-blind baseline should suffer most.

B. The force interface (Q1)

A force channel is worth having only if it is *free when unused* and *effective when used*. We test the two halves separately against SONIC.

(a) Free when unused. We give the low-level a zero force command and no external load and measure how well it tracks (Table II). Adding the channel costs nothing: whole-body posture is comparable between the two trackers, our forced-body error is about half SONIC's, and we finish every episode. SONIC's completion shortfall is concentrated on clips whose reference travels, where its lack of a root-position input costs it the episode before any load is applied.

(b) Effective when used. This test asks whether being told about a load helps the robot survive it. An external wrench is applied to the robot's loaded bodies, as if it were carrying something, and the robot's only way to keep its reference posture is to push back with an equal and opposite force. Each arm is simultaneously *told* what that wrench is through the force command; the command is precisely the information needed to push back correctly (Fig. 3). The wrench is recorded once per clip and replayed identically to every arm, scaled together with the command. The arms therefore differ only in what they are told: ours, the same policy with the command hidden, the same policy with the command's sign inverted, and SONIC, with no such input at all (Fig. 5). An arm that ignores the command must resist a load it cannot anticipate; an arm that uses it can pre-load against it.

The four arms separate immediately and stay ordered. Ours degrades slowly, holding 75% completion at full load where the other three hold 30% or less, and its forced-body error stays within a few centimetres while theirs roughly triples. That the sign-inverted arm is the worst throughout is the evidence that the channel is consulted rather than decorative: a policy ignoring its command would be indifferent to the sign. The walking clips are the weakest for every arm, and there SONIC fails before any load is applied.

C. Task performance under increasing load (Q2)

We pin object mass to a multiple of each object's nominal value and measure how the stack degrades with the force channel open and closed (Fig. 6), averaged over the hand and rope tasks.

The channel does not shift a steady decline; it changes the shape of the failure. The force-blind arms hold and then drop. SONIC completes 40% at half nominal mass and still 40% at three quarters of it, 31% at nominal, and 1% by 1.25×; our own policy with the channel closed runs 66, 60, 43, 16 and 0 across the same five points. Each gives way within a quarter of nominal, just past the mass it was trained around. Ours stays above the 90% line across the swept range, which neither force-blind arm reaches anywhere in it, not even at half nominal mass, and object error separates in the same direction.

The two force-blind arms share no weights, one being an external tracker and the other our policy with the command removed. What they have in common is the missing command rather than a shared weakness in training. Neither reads measured force, here or anywhere in this paper. The channel widens the operating range rather than removing the mass limit: swept further than the figure shows, ours gives way too.

D. Online force adaptivity (Q3)

The previous section changed mass between episodes. Here we change it within one episode and ask whether the commanded force follows, measuring the adaptation gain, the change in commanded force per kilogram, after the object settles. The policy observes neither object mass nor measured force.

The commanded force follows the load with the correct sign in every task and both directions, and at close to the physically required magnitude (Fig. 7): on the tethered clips the gain is 9.4–10.3 N/kg against the 9.81 a suspended mass requires, and on the sliding clips it is what μg predicts, though the response is weaker on release than on loading.

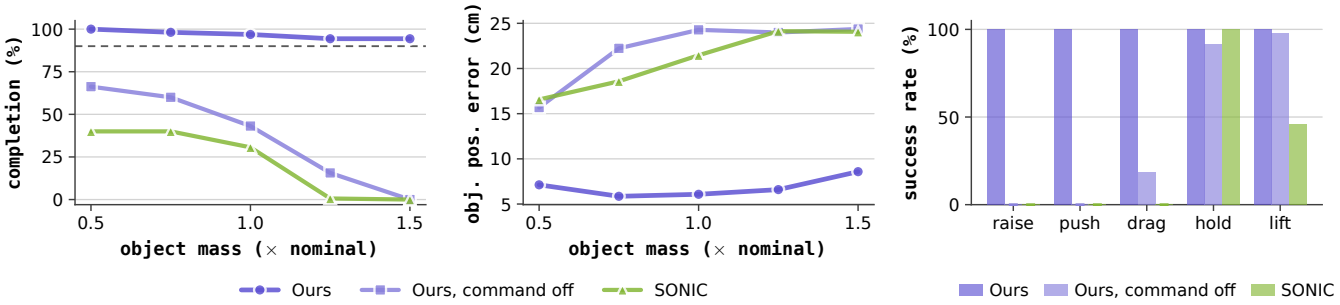


Fig. 6. Task performance as object mass increases, averaged over the five hand and rope tasks. Nominal is each object's trained-band midpoint, so 1.0× is the same relative position in every band. Completion is over all active steps; an arm terminating early is read together with its survival. The dashed line is 90% completion. *Right:* success at nominal mass, per task, for the hand policy; the tethered tasks have mirrored left and right clips and we draw the right one of each. The push bar uses a different criterion, surviving four seconds rather than finishing the clip, because every arm fails the clip there and the criterion would hide that the force-blind arms terminate after roughly two seconds while ours runs on.

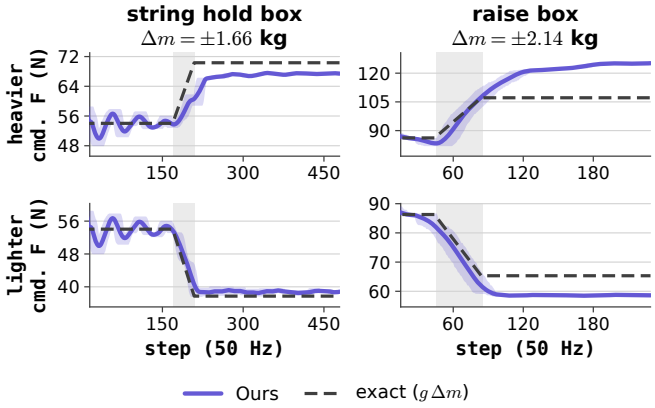


Fig. 7. Commanded force through an in-episode mass step on a static hold, summed over force bodies (`q3_adaptivity`). The line is a moving average and the shaded band is the spread it removed, the minimum and maximum within the same window; the grey column is the mass ramp. The high-level force planner observes neither mass nor measured force. A tethered box is an undamped pendulum, and both commanders' force oscillates with the swing; the analytical force model oscillates at least as much as the learned policy; the trend rather than the raw trace therefore carries the reading.

TABLE III. Command comparison over the five interaction tasks. 3-D object position error in cm, completion in % in parentheses. "no cmd." is our policy with the force command removed; "BC only" is the same architecture trained by behavior cloning alone. Where a task has mirrored left and right clips we report the side our own policy does worse on.

Task	no cmd.	BC only	Ours
Raise box	19.45 (0)	26.03 (6)	**4.59** (100)
String lift box	25.25 (48)	23.65 (0)	**6.26** (100)
String hold box	26.40 (75)	21.05 (84)	**6.28** (100)
Push box	29.45 (0)	24.80 (0)	**4.91** (84)
Drag box	32.05 (19)	**5.33** (100)	6.95 (100)

E. Why both imitation and reinforcement (Q4)

(a) Why reinforcement is needed. The high-level force planner is cloned from an analytical force model that reads the object's true mass and inertia, which no deployed policy has. The question is how much of it survives distillation into the observations our policy actually receives. We compare behavior cloning alone against our full recipe and against the same policy with the force command removed (Table III).

Imitation alone does not survive the change of observations: on four of the five tasks the cloned policy lands near the no-command arm, asking for forces the contact never delivers, because the signal that would say so is the privileged input it no longer has. Adding the reinforcement objective, scored on the object rather than on agreement with the teacher, recovers all five. The drag box is the exception, where a constant pull learned from the teacher is already close to optimal because a towed crate charges only friction per unit weight. BC-only therefore performs comparably on this task: while it achieves a slightly lower object error than ours, both policies reach 100% success rate.

(b) Why imitation is needed. The reverse question does not admit a controlled comparison: a policy trained without the analytical force model fails differently from run to run. We report instead what recurred across our attempts. On clips that alternate between loaded and unloaded phases, reinforcement learning alone collapses the distinction, commanding force throughout or not at all, and which of the two it settles on depends on how the force samples are drawn. On clips that are loaded from beginning to end it often reaches acceptable task performance while commanding forces that bear little relation to what the mechanics require. The analytical force model supplies a pattern rather than a score: cloning it gives the policy the shape of the force profile, when to load and roughly how much, and leaves reinforcement learning to refine a signal whose coarse structure is already right. Training is also markedly cheaper.

F. Transfer to hardware (Q5)

We deploy the hierarchy on a Unitree G1 with onboard sensing only and object pose from AprilTag markers, and run it on four interaction tasks and one composite: dragging a cart, pushing a box, raising a bin, kicking a box, and the raise-walk-kick sequence.

Throughout the evaluation, we compare against SONIC using the same hardware, objects, and reference motions. (Fig. 8). Our deployment experiments address two questions. First, can the interface transfer successfully to the real system? We evaluate this through the task experiments. Second, is the adaptation observed in simulation attributable to the high-level force planner? We examine this through a three-

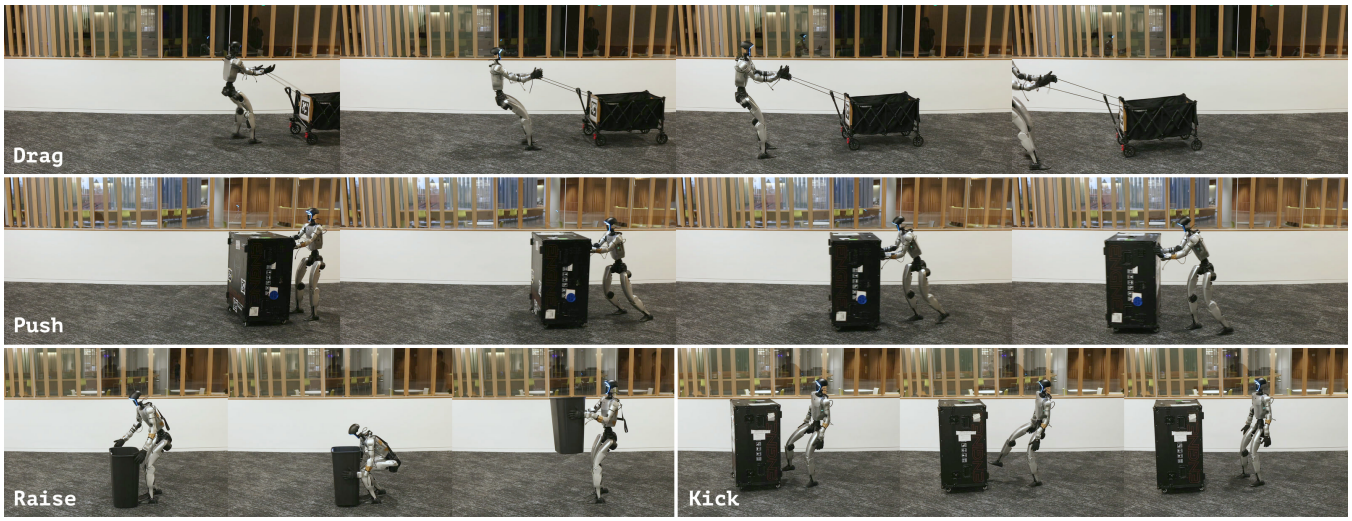


Fig. 8. Hardware deployment on a Unitree G1 using onboard sensing and AprilTag-based object pose estimation. From top to bottom: dragging the cart, pushing the box, and raising the bin followed by kicking the box. The pushed and kicked boxes reach 20 kg, the heaviest load tested.

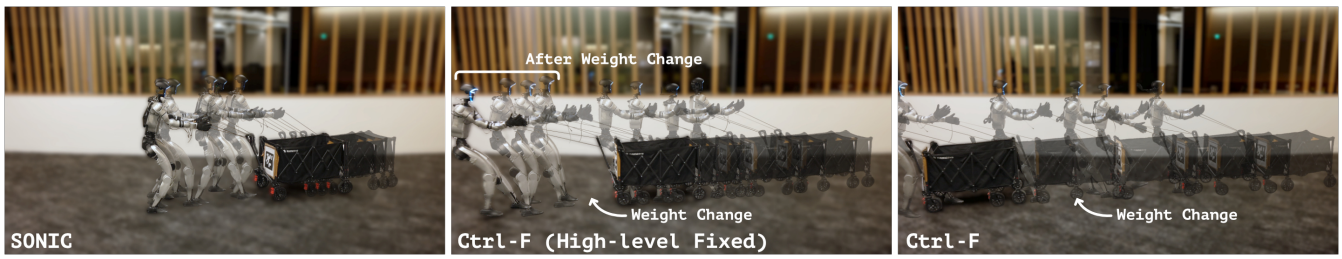


Fig. 9. Isolating the high-level force planner on hardware. Every arm is loaded while already dragging the cart, whose wheels stay locked throughout (arrow marks the loading). Frames are composited at equal time intervals; the spacing between poses is therefore speed. SONIC is given the easier test, an empty cart that takes 8 kg, because it cannot start a loaded cart moving at all; it is stopped nonetheless. Our two arms start at 8 kg and take a further 6 kg. Under a command frozen at what the high-level force planner itself issued for the initial load, the robot keeps walking but slows, its poses bunching together after the change, while the full hierarchy holds its pace. Those two share a tracker, an interface and a starting force command, and differ only in whether that command is allowed to change.

arm loading test.

Hand tasks. We deploy three: dragging a cart, pushing a box, and raising a bin. The cart's wheels are locked so the robot drags it across carpet rather than rolling it, making its load sliding friction. SONIC moves the empty cart and nothing heavier, stepping in place under any added weight; ours drags it empty, with 8 kg and with 14 kg, and fails at 20. On the push box, SONIC does not move it either empty or loaded with 20 kg, while ours pushes both at no obvious difference in speed, the behavior a command that scales with the object should produce. On the bin, SONIC raises the empty one because its hands catch the rim and fails with 2 kg inside, its hands sliding past: nothing presses inward, and the lip alone will not hold the load. Ours raises both, commanding an inward force whose normal pressure is what produces grip.

Foot tasks. We deploy the kick. SONIC cannot perform it at all once the object resists, being unstable on one leg; ours kicks the box both empty and with 20 kg in it.

Composite task. We deploy raise-walk-kick, which loads the hands and an ankle within one motion: the robot carries a 1 kg box while kicking a cart loaded with 8 kg, and performs the sequence successfully.

Mid-episode loading. The tasks above show that the interface transfers, not which level adapts: a tracker that merely tolerates load would look much the same. We therefore load the cart mid-drag and freeze the high-level force planner on one arm, dropping it two seconds in and holding its command at the median of what it had just issued (Fig. 9). That arm keeps our tracker, our interface and the command our own policy chose for the load it could see, and lacks only permission to revise it. It slows once the weight goes in and the full hierarchy does not; the adaptation therefore belongs to the high-level force planner and not to the tracker. This is the in-episode adaptation of Sec. IV-D on hardware, with no force sensor in the loop.

V. LIMITATIONS

The high-level force planner requires object-state estimates, obtained on hardware from AprilTag markers. This introduces deployment overhead and makes the system dependent on reliable marker detection. Future work may explore using depth image as object observation.

Moreover, our policies are trained on a limited set of motions. Although a single policy supports diverse interaction mechanics, including unilateral pushing, frictional grasping,

rope tension, and impact, it does not yet generalize to unseen motions. Unlike general motion trackers, the policy cannot follow a new motion clip without retraining. Scaling this interface to a large motion corpus is therefore a natural next step toward a general motion–force controller.

VI. CONCLUSION

We presented a hierarchy framework for humanoid object interaction that explicitly controls the robot's interaction with the environment through force. Inverse-force training equips a general motion tracker with explicit force commands at arbitrary contact bodies without requiring objects during training. Above the frozen tracker, a high-level force planner specifies force commands from object and robot state in a closed-loop manner. The planner is supervised by an analytical force model that unifies four interaction mechanics and further refined through reinforcement learning. The analytical force model uses privileged states only during training; deployment requires only the tracker and learned force planner.

In simulation, the interface costs little tracking when idle, holds its reference under an external load by exerting back the opposing force it is commanded, and adapts online to mass changes it never observes. It holds completion above 90% out to $1.5\times$ nominal object mass, where both force-blind arms have fallen to zero; at nominal mass those two stand at 43% and 31%. Through an in-episode mass step the commanded force follows at 9.4–$10.3\,\mathrm{N/kg}$ against the 9.81 a suspended mass requires. On hardware the same policy carried, dragged, pushed and kicked loads the force-blind baseline could not move at all, and absorbed weight added in the middle of a drag. Future work includes richer object classes and an object representation that needs no marker.

REFERENCES

[1] X. B. Peng, P. Abbeel, S. Levine, and M. van de Panne, "DeepMimic: Example-guided deep reinforcement learning of physics-based character skills," *ACM Trans. Graph.*, vol. 37, no. 4, Art. no. 143, 2018.

[2] N. Mahmood, N. Ghorbani, N. F. Troje, G. Pons-Moll, and M. J. Black, "AMASS: Archive of motion capture as surface shapes," in *Proc. IEEE/CVF ICCV*, 2019, pp. 5442–5451.

[3] Q. Liao, T. E. Truong, X. Huang, Y. Gao, G. Tevet, K. Sreenath, and C. K. Liu, "BeyondMimic: From motion tracking to versatile humanoid control via guided diffusion," *Science Robotics*, vol. 11, no. 117, Art. no. eadx8924, 2026.

[4] Z. Luo, Y. Yuan, T. Wang, *et al.*, "SONIC: Supersizing motion tracking for natural humanoid whole-body control," *Science Robotics*, vol. 11, no. 117, Art. no. eaed4592, 2026.

[5] Z. Chen, M. Ji, X. Cheng, X. Peng, X. B. Peng, and X. Wang, "GMT: General motion tracking for humanoid whole-body control," in *Proc. IEEE/RSJ IROS*, 2026.

[6] Z. Qi, X. Chen, J. Wang, C. Lin, Y. Lian, W. Zhang, X. Yu, H. Wang, and L. Yi, "Humanoid generative pre-training for zero-shot motion tracking," in *Proc. IEEE/CVF CVPR*, 2026, pp. 20834–20844.

[7] T. He, Z. Luo, W. Xiao, C. Zhang, K. Kitani, C. Liu, and G. Shi, "Learning human-to-humanoid real-time whole-body teleoperation," in *Proc. IEEE/RSJ IROS*, 2024, pp. 8944–8951.

[8] H. Weng, Y. Li, N. Sobanbabu, Z. Wang, Z. Luo, T. He, D. Ramanan, and G. Shi, "HDMI: Learning interactive humanoid whole-body control from human videos," arXiv:2509.16757, 2025.

[9] S. Chen, S. Zhao, Z. Wu, J. Li, G. Shi, and C. K. Liu, "SceneBot: Contact-prompted general humanoid whole body tracking with scene-interaction," arXiv:2606.27581, 2026.

[10] Y. Lin, J. Cui, Y. Li, B. Jia, Y. Zhu, and S. Huang, "LessMimic: Long-horizon humanoid interaction with unified distance field representations," arXiv:2602.21723, 2026.

[11] N. Hogan, "Impedance control: An approach to manipulation, Part I—Theory," *J. Dyn. Syst. Meas. Control*, vol. 107, no. 1, pp. 1–7, 1985.

[12] M. H. Raibert and J. J. Craig, "Hybrid position/force control of manipulators," *J. Dyn. Syst. Meas. Control*, vol. 103, no. 2, pp. 126–133, 1981.

[13] Y. Zhang, Y. Yuan, P. Gurunath, I. Gupta, S. Omidshafiei, A. Aghamohammadi, M. Vazquez-Chanlatte, L. Pedersen, T. He, and G. Shi, "FALCON: Learning force-adaptive humanoid loco-manipulation," in *Proc. Learning for Dynamics and Control Conf. (L4DC)*, 2026.

[14] S. Chen, Z.-A. Cao, Z. Luo, F. Castañeda, C. Li, T. Wang, Y. Yuan, L. Fan, C. K. Liu, and Y. Zhu, "CHIP: Adaptive compliance for humanoid control through hindsight perturbation," arXiv:2512.14689, 2025.

[15] P. Zhi, P. Li, J. Yin, B. Jia, and S. Huang, "Learning a unified policy for position and force control in legged loco-manipulation," in *Proc. Conf. on Robot Learning (CoRL)*, 2025, pp. 652–669.

[16] A. Rajeswaran, V. Kumar, A. Gupta, G. Vezzani, J. Schulman, E. Todorov, and S. Levine, "Learning complex dexterous manipulation with deep reinforcement learning and demonstrations," in *Proc. Robotics: Science and Systems XIV*, 2018.

[17] S. Ross, G. Gordon, and D. Bagnell, "A reduction of imitation learning and structured prediction to no-regret online learning," in *Proc. AISTATS*, 2011, pp. 627–635.

[18] K. Zakka, Q. Liao, B. Yi, L. Le Lay, K. Sreenath, and P. Abbeel, "mjlab: A lightweight framework for GPU-accelerated robot learning," arXiv:2601.22074, 2026.